

C# JWT Token Generation and Validation

<https://gemini.google.com/app/a9289dd7e359042d>

User prompt: Generate me a jwt token service in c# that i can use to generate jwt token, validate jwt token. I want to use the validation on certain endpoints

Response: To set up a complete JWT authentication flow in ASP.NET Core, you need a service to generate the tokens and a middleware configuration to automatically validate them on your endpoints.

Here is the complete implementation.

1. Install Required Packages

First, ensure you have the necessary NuGet package installed in your project:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

2. Add JWT Configuration (appsettings.json)

Add your JWT settings to your configuration file. **Important:** The key must be at least 32 characters long for the HMACSHA256 algorithm to work properly.

```
{
  "Jwt": {
    "Key": "YourSuperSecretKeyThatIsAtLeast32BytesLong!",
    "Issuer": "YourIssuer",
    "Audience": "YourAudience",
    "ExpireMinutes": 60
  }
}
```

3. Create the JWT Service

This service handles generating tokens and provides a manual validation method (though ASP.NET Core will handle the standard endpoint validation automatically).

IJwtService.cs

```
using System.Security.Claims;

public interface IJwtService
{
    string GenerateToken(string userId, string username, string role);
    ClaimsPrincipal? ValidateToken(string token);
}
```

JwtService.cs

```
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.IdentityModel.Tokens;

public class JwtService : IJwtService
{
    private readonly IConfiguration _configuration;

    public JwtService(IConfiguration configuration)
    {
        _configuration = configuration;
    }

    public string GenerateToken(string userId, string username, string role)
    {
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));
        var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
```

```

var claims = new[]
{
    new Claim(JwtRegisteredClaimNames.Sub, userId),
    new Claim(JwtRegisteredClaimNames.Name, username),
    new Claim(ClaimTypes.Role, role),
    new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
};

var tokenDescriptor = new SecurityTokenDescriptor
{
    Subject = new ClaimsIdentity(claims),
    Expires = DateTime.UtcNow.AddMinutes(double.Parse(_configuration["Jwt:ExpireMinutes"]!)),
    Issuer = _configuration["Jwt:Issuer"],
    Audience = _configuration["Jwt:Audience"],
    SigningCredentials = credentials
};

var tokenHandler = new JwtSecurityTokenHandler();
var token = tokenHandler.CreateToken(tokenDescriptor);

return tokenHandler.WriteToken(token);
}

public ClaimsPrincipal? ValidateToken(string token)
{
    var tokenHandler = new JwtSecurityTokenHandler();
    var key = Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!);

    try
    {
        tokenHandler.ValidateToken(token, new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(key),
            ValidateIssuer = true,
            ValidIssuer = _configuration["Jwt:Issuer"],
            ValidateAudience = true,
            ValidAudience = _configuration["Jwt:Audience"],
            ClockSkew = TimeSpan.Zero // Tokens expire exactly at token expiration time
        }, out SecurityToken validatedToken);

        var jwtToken = (JwtSecurityToken)validatedToken;

        // Return the claims principal if you need to read the claims manually
        return new ClaimsPrincipal(new ClaimsIdentity(jwtToken.Claims, "jwt"));
    }
    catch
    {
        // Validation failed (expired, tampered, invalid signature, etc.)
        return null;
    }
}
}

```

4. Configure Authentication (Program.cs)

To protect specific endpoints automatically, you need to register the authentication middleware and the JWT bearer configuration.

```

using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;

var builder = WebApplication.CreateBuilder(args);

```

```
// 1. Register the JWT Service for Dependency Injection
builder.Services.AddScoped<IJwtService, JwtService>();

// 2. Configure JWT Authentication
var jwtSettings = builder.Configuration.GetSection("Jwt");
var key = Encoding.UTF8.GetBytes(jwtSettings["Key"]!);

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = jwtSettings["Issuer"],
        ValidAudience = jwtSettings["Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(key),
        ClockSkew = TimeSpan.Zero
    };
});

builder.Services.AddControllers();

var app = builder.Build();

// 3. Add Authentication and Authorization middlewares (Order matters!)
app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
app.Run();
```

5. Protect Your Endpoints

Now you can use the `[Authorize]` attribute to enforce token validation on specific controllers or actions. ASP.NET Core will automatically intercept the request, extract the token from the `Authorization: Bearer <token>` header, and validate it.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IJwtService _jwtService;

    public AuthController(IJwtService jwtService)
    {
        _jwtService = jwtService;
    }

    // Unprotected endpoint: Anyone can call this to get a token
    [HttpPost("login")]
    [AllowAnonymous]
    public IActionResult Login()
    {
        // TODO: Validate user credentials against your database here

        // Generate the token
        var token = _jwtService.GenerateToken("123", "johndoe", "Admin");
    }
}
```

```
        return Ok(new { Token = token });
    }

    // Protected endpoint: Only requests with a valid JWT token can access this
    [HttpGet("secure-data")]
    [Authorize]
    public IActionResult GetSecureData()
    {
        // You can access the claims from the validated token like this:
        var username = User.Identity?.Name;
        return Ok(new { Message = $"Hello {username}, you have accessed a protected route!" });
    }

    // Role-based protected endpoint
    [HttpGet("admin-only")]
    [Authorize(Roles = "Admin")]
    public IActionResult GetAdminData()
    {
        return Ok(new { Message = "This is classified admin data." });
    }
}
```
